

응용-D1 · GEMINI CLI · VS CODE 강의 교안

Coding Agent 실전: AI기반 TDD 마스터 과정

개념에서 코드까지, 추적성을 잃지 않는 개발

4계층 검증 구조

계약 ID

추적성 매트릭스

Dual-Track TDD $\equiv$ ARRR

하네스 엔지니어링

하루 8시간 집중 과정 · 08:30 – 17:30 | 강의 40% + 실습 60% | 개발자 및 입문자 대상

2026 · 김대경 Ph.D.

과정 개요 — 하루 만에 C2C 개발 흐름을 체득한다

항목	내용
교육 대상	개발자 및 입문자 (Git 기초 숙지 전제)
교육 시간	8시간 — 08:30~17:30 (점심 11:45~13:00)
교육 방식	강의 40% + 실습 60% (개념→시연→개별 실습)
도구 스택	VS Code · Gemini CLI · Gemini Code Assist
사전 준비	Node.js 20+ · Python 3.11+ · Git/GitHub · Google 계정

관통 예제 — 장바구니 할인 계산기

계약 ID 6개를 ARRR 사이클로 완성

INV-1

INV-2

INV-3

INV-4

E-1

E-2

핵심 목표

개념이 코드까지 추적되는(C2C) 개발 흐름을 1일 안에 체득한다

오늘의 산출물 — docs/PRD.md(계약 SSOT) · tests/(계약 ID가 박힌 테스트) · src/(ID 주석이 달린 구현) · docs/traceability.md(추적성 매트릭스)

이 과정을 관통하는 하나의 생각 — 인과 사슬

① 목표

C2C (개념→코드 추적성)

요구·개념이 코드까지 추적 가능해야 한다. 이것이 전부 이유다.

② 왜

추적성(Traceability) 확보

테스트 없는 코드는 “어디가 왜 깨졌는지” 모른다. 추적성이 이 문제를 푼다.

③ 수단 = 사이클

Dual-Track TDD ≡ ARRR

ARRR를 돌리는 것이 곧 Dual-Track TDD 실행이고, 그것이 추적성 확보다.

④ 결과

추적 가능한 Clean Code

추적 가능하고, 테스트가 보호하며, 계속 손볼 수 있는 깨끗한 코드.

Ask = RED · Respond = GREEN · Refine = REFACTOR · Repeat = 다음 계약 ID로 다음 질문

강사 한 마디 — “ARRR”과 “C2C × Dual-Track TDD”는 따로가 아닙니다. 같은 것의 두 이름입니다. 이 한 줄을 새기고 가면 오늘은 성공입니다.

학습 목표 — 오늘 가져갈 6가지

1 C2C 정식화

4계층 검증 구조 위에서 불변식·에러 계약에 ID를 부여하고, 테스트·구현·커밋까지 끊기지 않게 잇는다

2 제품 계약 (상류)

Mom Test 고객 증거를 INV/E/AC + 증거 수준 (L0~L2)으로 형식화해 불변식의 원천을 검증한다

3 Dual-Track TDD $\equiv$ ARRR

Boundary(Track A)와 Entity(Track B)를 나눠 Ask→Respond→Refine→Repeat를 돌린다

4 하네스 엔지니어링

프롬프트→컨텍스트→하네스 3수준으로 AI 유발 추적 단절(과잉 구현·테스트 약화)을 예방한다

5 AARRR 계측 (하류)

성공 기준을 계측 코드로 구현해 “코드가 개념을 실증했는가”를 운영 데이터로 귀환시킨다

6 추적성 매트릭스

계약↔테스트↔구현의 1:1 대응을 검사 가능한 산출물로 만들고 “정체불명 코드”를 제거한다

① · ③ · ⑥은 C2C의 골격, ②는 상류(개념의 원천), ⑤는 하류(개념의 실증), ④는 AI 시대에 그 사슬을 지키는 방어선입니다.

사전 준비 — 준비물 점검

0.1 준비물 점검 — 시작 전에 갖춰야 할 것

개인 PC

Windows · macOS · Linux 모두 가능 (듀얼 모니터 권장)

GitHub 계정

없다면 github.com 에서 무료 가입

Google 계정

Gemini CLI · Code Assist 로그인 — 무료 등급 사용 가능

Node.js 20 이상

Gemini CLI 설치에 필요 — nodejs.org 에서 LTS

Python 3.11 이상 · Git

pytest · Flask 실습과 커밋 규율에 사용

버전 확인 — 이 3줄이 통과해야 시작

```
$ node --version  
v20.x.x ✓ (20 이상)
```

```
$ python --version  
Python 3.11.x ✓
```

```
$ git --version  
git version 2.x ✓
```

```
$ gemini --version  
↳ 아직 없다면 0.2에서 설치
```

△ Node.js가 20 미만이면 Gemini CLI 설치가 실패합니다. 가장 흔한 사전 준비 실패 원인입니다 — 반드시 먼저 확인하세요.

도구 스택과 설치 — 세 가지의 역할 분담

VS Code

에디터 + 통합 터미널

코드 열람·수동 편집, 터미널에서 gemini-pytest 실행

Gemini CLI

터미널 AI 에이전트

다중 파일 편집·테스트 자동 실행·자율 반복 —
ARRR의 주력 도구

Gemini Code Assist

VS Code 확장 (채팅·Agent)

코드 질의(Ask)·인라인 편집·Agent 탭 (CLI와 동
일 엔진)

```
# 1) VS Code 설치: code.visualstudio.com → 확장에서 "Gemini Code Assist" 설치
# 2) Gemini CLI 설치 (Node.js 20+ 필요)
npm install -g @google/gemini-cli
# 3) 실행 및 로그인 (Google 계정, 무료 등급 가능)
gemini      # 첫 실행 시 브라우저 로그인 → 대화형 세션 시작
            # /model 로 Gemini 2.5 Pro 권장 (추론에 강함)
```

CLI와 Code Assist는 같은 작업의 두 얼굴입니다 — 이 워크북의 ▶ 프롬프트 박스는 gemini 세션과 Code Assist Agent 탭 어느 쪽에 넣어도 동일하게 동작합니다.

Gemini CLI 기본기 — 핵심 진입점 5가지

진입점	무엇 · 언제
gemini	대화형 세션 — 자연어로 지시 (모든 ARRR 작업)
gemini -p "..."	단발 실행 후 종료 — 스크립트·파이프라인용
@경로	파일·폴더를 컨텍스트에 정확히 지정 (@src/cart.py)
!명령	세션 안에서 셸 직접 실행 (!pytest -q)
/명령	내장·커스텀 슬래시 명령 (/model · /memory · /tdd-red)

전체 슬래시 명령·플래그·단축키는 워크북 부록 B(레퍼런스)에 정리돼 있습니다.

실습 0-1 — 첫 대화

- gemini-practice 연습용 폴더를 만들고 그 안에서 gemini를 실행한다.
- "안녕, 너는 무엇을 도와줄 수 있어?" 라고 물어보고 답을 읽어 본다.
- "hello.txt 파일을 만들고 'Hello, Gemini CLI'라고 써줘" 를 요청한 뒤, 승인 단계에서 동작을 관찰한다.
- /quit 으로 세션을 종료한다.

실습 0-2 — @ 와 !

- @sample.py 이 코드를 설명해줘
- !ls · !pytest -q 실행
- 빈 입력에서 ! → 셸 모드 토글

승인 모드 — plan(읽기 전용) · default(매번 승인, 권장 시작점) · auto_edit(편집만 자동) · yolo(전부 자동, 격리 저장소에서만). Shift+Tab 으로 전환됩니다.

소프트웨어 1.0 · 2.0 · 3.0 — 코딩 방식의 진화



소프트웨어 3.0 — 프롬프트가 곧 프로그램. 이 과정은 그 위에 “계약·추적성”을 얹어, 프롬프트로 짠 코드의 통제력을 지킨다.

문제의식 — 바이브 코딩과 C2C 단절

바이브 코딩 (Vibe Coding)

AI에게 즉흥적 요청을 반복해 기능을 얻어내는 방식. 동작은 하지만 “이 코드가 어느 요구·개념에서 나왔는가”를 역추적할 수 없다 = C2C 단절

- 요구 만족을 객관적으로 보여 줄 근거가 없다
- 무엇을 왜 고쳤는지 나중에 추적할 수 없다
- 에이전트가 오래 일할수록 “통과한 것처럼 보이는” 코드가 는다

핵심 — 테스트는 가장 강력한 검증 수단

통과하는 테스트 묶음 = 세션이 아무리 길어져도 변하지 않는 외부의 진실.

켄트 벡: AI 에이전트와 일할 때 오히려 예전보다 TDD를 더 많이 한다.

개발자의 역할 이동: 코드 작성자 → C2C 추적성의 설계자·감리자.
계약 표 · 골든셋 · 실패 테스트 · 규칙 파일이 명시적 산출물이 된다.

미니 체험 ① (15분) — 바이브로 만든 코드, 추적 가능한가?

① gemini 실행 → “add 함수 만들어줘” ② “음수도”, “실수도” 즉흥 요청 2~3회 ③ 자문: “이 코드는 어떤 요구에서 나왔나? 무엇이 보장되나? 테스트는?” → 개념→코드 연결고리가 없음을 확인

AI 페어는 계약→구현 구간의 처리량을 높이지만, 동시에 추적 단절의 최대 위험원입니다. 생산성과 추적성은 자동으로 함께 오지 않습니다.

미니 체험 ② — 같은 기능을 TDD로 재구현

RED — 의도된 실패

프롬프트

"tests/test_calc.py 에 test_add 테스트만 작성해줘. src/calc.py 는 만들지 마."

멘트

"add 함수가 아직 없으니 pytest를 돌리면 의도된 실패(RED)가 납니다."

GREEN — 최소 통과

프롬프트

"방금 테스트를 통과시키도록 src/calc.py 에 add(a,b)를 가장 단순하게 구현해줘."

멘트

"1 passed — 이 테스트는 이제 우리 코드의 안전자산이 되었습니다."

REFACTOR — 안전한 개선

프롬프트

"불필요한 주석·가독성 저해 요소를 정리해줘. 정리 후에도 pytest 통과를 확인해줘."

멘트

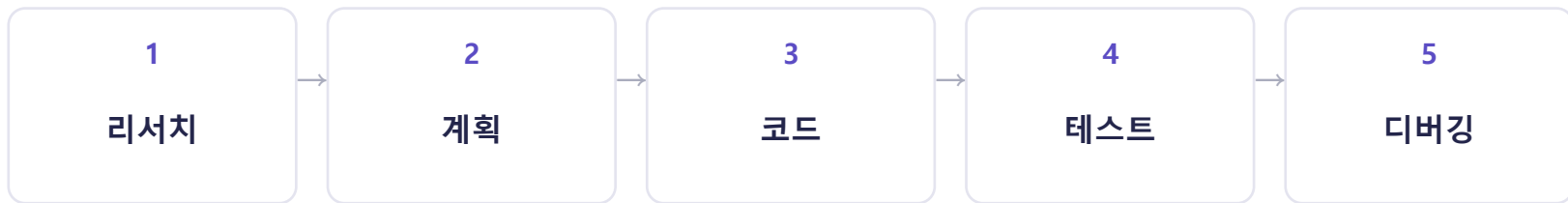
"리팩토링해도 여전히 1 passed — 테스트가 코드를 지켜주기에 불안하지 않다."

 미니 실습 — subtract 를 직접 | 1-A 바이브(5분): "subtract 추가해줘" → 테스트 없음 확인 | 1-B TDD(12분): RED → GREEN(return a-b) → REFACTOR + COMMIT

비교 질문 — "구현부터 한 1-A와, 실패를 보고 한 단계씩 밝은 1-B 중 어느 쪽이 더安心이 되나요?" 이 체감이 오늘 하루의 출발점입니다.

개발의 다섯 단계 — 리서치에서 디버깅까지

에이전트와 함께하는 개발도 이 다섯 단계를 지난다 — 관건은 각 단계를 계약과 커밋으로 “추적 가능”하게 남기는 것(C2C).



바이브 코딩은 이 흐름을 “한 줄 요청”으로 뭉개기 쉽다 — 그래서 계약·테스트·커밋으로 각 단계를 다시 세운다.

바이브 코딩 vs 에이전틱 엔지니어링

바닥을 높이는 거 vs 천장을 확장하는 거

vibe coding

↑ 바닥을 높인다

“이런 앱이 있으면 좋겠다”는 한 줄에서 진짜 앱이 나온다. 모두를 위해 진입 장벽을 낮춘다.

agentic engineering

↑ ↑ 천장을 확장한다

프로페셔널 소프트웨어의 품질 기준을 그대로 유지하면서, 더 빠르게 나아간다.

책임은 여전히 본인한테 있다

생각하기 vs 이해하기 — 무엇을 아웃소싱할 수 있나

Thinking — 사유

정보를 받아 분석하고, 단계를 밟아 결론에 도달하는 처리 작업.

✓ LLM이 잘함 — 아웃소싱 가능

Understanding — 이해

정보가 자기 모델 안에서 자리 잡고, 다른 지식과 연결되어 “깨달음”으로 바뀌는 것.

✗ 본인이 직접 — 아웃소싱 불가

핵심 — 생각(Thinking)은 위임해도 이해(Understanding)는 위임할 수 없다. 이해를 아웃소싱하면 코드의 통제력을 잃는다. (증강 코딩: “씨앗을 먹지 마라”)

C2C의 정의와 4계층 검증 구조

C2C(Concept-to-Code Traceability) — “무엇이 절대 바뀌면 안 되는가”라는 개념. 불변식에서 출발해 코드까지 연결고리를 끊지 않는 것. 임의의 구현 줄에서 위로 올라가면 반드시 불변식에, 임의의 불변식에서 아래로 내려가면 테스트와 구현 줄에 닿는다 — 이 양방향 도달 가능성이 곧 C2C.

계층	이름	질문	ECB 대응
Layer 1	Invariant (불변식)	무엇이 절대 바뀌면 안 되는가	수학적 진실
Layer 2	UI Contract Test	UX가 올바르게 반응하는가	Boundary
Layer 3	Domain Test	비즈니스 규칙이 작동하는가	Control
Layer 4	Implementation Unit	단위 알고리즘이 정확한가	Entity

⚠ ECB는 MVC·Spring Controller와 다르다. 분석 개념 도구이지, 코딩 규칙이 아니다 (Ivar Jacobson, OOSE 1992)

표 1. C2C 4계층 검증 구조 (논문 표 1) — 각 계층의 질문이 그 아래 계층의 검증을 규정합니다. 검증은 위에서 아래로 내려갑니다.

Layer 1(불변식)은 사슬의 정점 — 그런데 그 불변식은 어디서 오는가? 이 질문이 3장(상류 확장 · 제품 계약)으로 이어집니다.

계약 ID(추적의 못)와 추적성 매트릭스

RED

테스트 이름·주석에 계약 ID

GREEN

구현 줄마다 계약 ID 주석

COMMIT

커밋 메시지도 계약 ID

금지

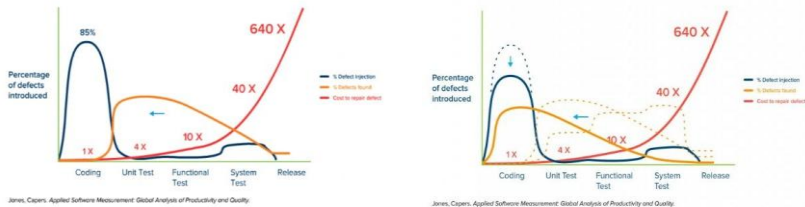
ID 없는 동작은 구현하지 않는다

계약 ID	의미	검증 테스트	구현 위치
INV-1	subtotal = $\sum \text{price} \times \text{qty}$	entity::test_subtotal_sums_price_times_qty	subtotal() return
INV-2	경계 포함 문턱 할인	entity::test_threshold_applies_at_50000 / _below	apply_threshold_discount()
INV-3	문턱→VIP 할인 순서·금액	entity::test_vip_order_and_value	final_total()
INV-4	$0 \leq \text{final} \leq \text{subtotal}$	entity::test_discount_never_increases_amount	final_total() return
E-1	None 입력 → TypeError	boundary::test_none_items_raises_type_error	subtotal() 가드
E-2	음수 → ValueError(index 포함)	boundary::test_negative_qty_reports_index	subtotal() 루프

구현에는 있으나 매트릭스에 없는 코드 줄 = “정체불명 코드” → 제거하거나 계약화합니다. 이 규칙이 켄트 벡이 꼽은 AI의 첫 번째 경고 신호(과잉 구현)를 원천 차단합니다.

Dual-Track TDD와 ARRR 실행 리듬

결함 비용 — Shift-Left Testing 결함 발견 시점이 늦을수록 수정 비용이 기하급수적으로 증가 → 초기 테스트(Shift-Left)가 핵심



단계	의미	TDD	완료 신호
A — Ask	테스트로 시스템에 질문한다	RED	의도된 FAIL
R — Respond	통과하는 최소 코드로 답한다	GREEN	전부 PASS
R — Refine	중복 제거·이름 정리로 다듬는다	REFACTOR	계약 불변 · GREEN
R — Repeat	다음 계약 ID로 다음 질문	다음 RED	회귀(pytest -q)

ARRR은 TDD의 RED→GREEN→REFACTOR를 대화적 리듬으로 재기술한 것입니다 — 새 기법이 아니라 같은 규율의 다른 이름입니다.

이중성 ① — 프로세스 수준

Discovery(무엇이 옳은가) ∥ Delivery(옳게 만든다).
Discovery의 계약·골든셋이 Delivery의 픽스처가 되고,
Delivery의 계측이 Discovery의 다음 가설이 된다.

이중성 ② — 테스트 아키텍처

Track A(UI/Boundary) ∥ Track B(Domain/Entity). 각 트랙에서 ARRR를 돌리며, 4계층의 Layer 2 와 Layer 3~4를 각각 담당한다.

△ 용어 주의 — 네 글자 ARRR(Ask-Respond-Refine-Repeat)은 실행 리듬, 다섯 글자 AARRR(Acquisition...Revenue)은 그로스 퍼널 지표. 별개의 개념입니다.

Dual-Track TDD 5단계 사이클 — 두 트랙의 맞물림

단계	Discovery 트랙	Delivery 트랙	ARRR	신호
① 가설 수립	계약(INV/E/AC)·골든셋·목표 수치 정의, ID 부여	(대기)	—	계약 표
② Red	가설을 검증 가능한 조건으로 구체화	ID마다 실패 테스트(tests/만, src/ 금지)	Ask	의도된 FAIL
③ Green	구현이 가설을 만족하는지 1차 확인	통과 최소 구현, 구현 줄에 계약 ID 주석	Respond	전부 PASS
④ Refactor	평가 기준이 정밀해질 지점 식별	중복 제거·SSOT 추출, 동작·계약 불변	Refine	여전히 GREEN
⑤ 합류/반복	골든셋·AARRR 지표로 전체 검증	E2E 스모크 + 다음 ID로 회귀 확인	Repeat	매트릭스 완성

표 3. Dual-Track TDD 5단계 사이클과 ARRR의 대응

닫힌 루프 — Discovery의 산출물이 Delivery의 테스트 픽스처가 되고, Delivery의 계측이 Discovery의 다음 가설이 됩니다. 이것이 프로세스 수준의 Dual-Track입니다.

ARRR 실증 예제 — 문자열 덧셈 계산기

장바구니(4장)에 들어가기 전, 더 작은 예제로 ARRR 네 박자를 끝까지 돌립니다. 특히 여기서만 R = Repeat(새 요구사항으로 다음 사이클)까지 봅니다 — 1.1의 미니 체험은 RED→GREEN→REFACTOR 한 바퀴였습니다.

A = Ask (RED)

테스트 3개를 먼저 — 빈 문자열 → 0
· 단일 숫자 · 콤마 두 숫자. 구현이 없으니 ImportError/실패.

```
def
test_empty_string_returns_zero():
    assert add("") == 0
```

R = Respond (GREEN)

통과시키는 가장 단순한 구현만. 과도한 추상화-미래 대비 코드 금지.

```
def add(numbers: str) -> int:
    if not numbers:
        return 0
    return sum(...)
```

R = Refine (REFACTOR)

동작은 그대로, 책임만 분리 —
_is_blank(검증) / _calculate_sum(계산).

```
def _is_blank(numbers): ...
def _calculate_sum(numbers): ...
→ 테스트는 여전히 전부 통과
```

R = Repeat

새 요구사항: 세미콜론 구분자 지원.
다시 Ask(실패 테스트)부터 →
_split_numbers 추출.

```
add("3;7") == 10
→ RED → GREEN → REFACTOR 한 바퀴 더
```

핵심 원칙 — RED → GREEN → REFACTOR 사이클을 절대 건너뛰지 않는다 · 테스트가 실패하는 것을 먼저 확인한 뒤에만 구현한다 · “최소한의 코드”만 쓴다 · 리팩터링은 외부 동작을 바꾸지 않는 선에서만 한다.

TDD · 리팩터링 · Tidy First — 계승하는 규율

단계	이때 만지는 것	끝나는 신호
RED	tests/ 만	테스트가 "의도대로" 실패
GREEN	src/ 만	테스트가 전부 통과
REFACTOR	src/ (필요 시 tests/)	여전히 통과 + 더 깔끔

코드 스멜	대표 처방
중복 코드	함수 추출 후 재사용
긴 함수	함수 추출(Extract Function)
불명확한 이름	이름 바꾸기(Rename)
매직 넘버 (50000, 0.9)	상수로 추출 (SSOT)

TDD 세 원칙

- ① 실패를 먼저 본다
- ② 최소로 통과시킨다 (과잉 구현 금지)
- ③ 통과 후 정리한다

Tidy First 3규칙

- ① 구조 변경과 동작 변경을 한 커밋에 섞지 않는다
- ② 둘 다 필요하면 구조 변경을 먼저
- ③ 전후로 테스트를 돌려 동작 불변 확인

본 방법론은 새 기법의 발명이 아니라 검증된 기법들의 목적론적 재조직입니다 — 각 단계와 커밋에 계약 ID를 결부시켜 추적성의 운반 수단으로 재해석합니다.

Augmented Coding — “씨앗을 먹지 마라”

“AI에 코딩을 무심코 맡기는 것은, 농부가 배고프다고 다음 해 심을 씨앗까지 먹어 치우는 것과 같다.” — 켄트 벅

	Vibe Coding	Augmented Coding
태도	돌아가면 OK, 결과를 거의 안 본다	주도권을 쥐고 결과에 책임진다
관심사	동작만	동작 + 테스트 + 설계 품질
AI와의 관계	맡기고 기다린다	감시하며 필요하면 개입·중단

△ AI가 결길로 새는 경고 신호

- ① 요청하지 않은 기능 — “합리적인 다음 단계”라도 시키지 않았으면 과잉 구현
→ “ID 없는 동작은 구현하지 않는다”가 차단
- ② 테스트를 속이는 정황 — 비활성화·삭제·수정으로 통과 시키기
→ “테스트 악화 금지”가 차단

이 두 경고 신호는 정확히 C2C 단절의 두 형태이며, 2장의 하네스에서 금지(Don't) 5조항으로 성문화됩니다 — 다섯 번째가 골든 마스터 보호 조항입니다

프로젝트 수준 — 13 STEP · 4개 페이즈 조감도

1 기획 · 아이템 선정

STEP 1~5

- 1 Self-Audit — 실행 가능 범위 확정
- 2-4 Job 검증 — JTBD · Mom Test · VPC
- 5 아이템 선택 — ICE 매트릭스로 1개

2 제품 설계 · 지표

STEP 6~8

- 6 제품 계약 — INV/E/AC + L0~L2 ●
- 7 AARRR 역산 설계 (6장)
- 8 PRD 확정 — 골든 시나리오 ●

3 엔지니어링 · 구현

STEP 9~12

- 9 대표 모듈 시연 — AARRR 완주 ●
- 10 하네스 구축 — pytest-GEMINI.md ●
- 11-12 모듈별 8단계 골격 반복 ●

4 평가 · 회고

STEP 13

- 13 골든셋 평가 · AARRR 실측
추적성 매트릭스 점검 · KPT ●

모듈 수준의 8단계 골격이 프로젝트 수준에서 13 STEP으로 확장됩니다. ● 표시가 오늘 실습 범위 — 나머지는 프로젝트 가이드(D2)의 Magic Square로 이어집니다.

이 워크북 사용법 — 반복되는 6개 박스

● A = Ask (RED)

계약을 검증하는 “실패하는 테스트”를 먼저. tests/ 만 만지고 src/ 는 건드리지 않는다. 의도된 FAIL이 완료 신호.

● R = Respond (GREEN)

실패 로그를 Context로 주고 “최소 구현”으로 통과. 구현 줄에 충족한 계약 ID를 주석으로 단다.

● R = Refine (REFACTOR)

전부 GREEN 상태에서 동작.계약을 바꾸지 않고 중복 제거-SSOT 추출. 구조/ 동작 커밋을 섞지 않는다.

▶ Gemini CLI 프롬프트

박스 안 문장을 gemini 세션에 그대로 붙여 넣고 Enter. Code Assist Agent 탭도 동일하게 동작.

▣ Context

“지금 상황”을 알려 주는 입력. 이 과정에서 가장 중요한 Context는 테스트 실패 로그입니다.

✓ 체크포인트 / ⚠ 자주 하는 실수

단계 완료 자가 점검 목록과, 많은 교육생이 막히는 지점-해결법. 막히면 부록 C(트러블슈팅)도 함께.

가장 중요한 Context는 테스트 실패 로그입니다 — 실패 로그를 주고 최소 구현을 요구하는 것이 Respond의 표준형이며, 이것이 과잉 구현을 억제합니다

실습의 뼈대 — Discovery → Delivery 8단계 골격

Discovery — 코드 작성 전에 끝낸다

① 목표 — 목표 표 1장

보장할 것 + ECB 역할·C2C 계층·트랙 좌표를 못 박는다

② PRD (실패 조건) — 실패 시나리오 표

기능 목록이 아니라 “무엇이 실패인가”를 L0~L3로 적는다

③ 계약 식별 + ID — 계약 표 (ID 완비)

불변식(INV-*)·에러 계약(E-*)에 ID 부여 — 추적의 못

④ 스켈레톤 — 의도적 미구현

시그니처 고정 + TODO(계약 ID). 완성 코드는 감춘다

Delivery — ⑤~⑧이 ARRR 사이클

⑤ A = Ask (RED) — 의도된 FAIL

ID마다 실패 테스트를 tests/에만 작성. src/ 수정 금지

⑥ R = Respond (GREEN) — 전부 PASS

실패 로그를 Context로 → 최소 구현. 구현 줄에 ID 주석

⑦ R = Refine (REFACTOR) — 여전히 GREEN

중복 제거·SSOT 추출. 동작·계약 불변

⑧ R = Repeat — 매트릭스 완성

다음 계약 ID로 다음 RED + 회귀(pytest -q)

절대 곧바로 코딩하지 않습니다. ④ 스켈레톤에서 완성 코드를 감추는 것은 교육 장치이자 추적성 장치 — 학습자와 AI 모두 “계약 → 테스트 → 구현 줄” 순서로만 코드에 도달합니다.

무엇을 만드나 — 장바구니 할인 계산기, 두 트랙으로

Track B

Logic — Entity

금액·할인 계산(순수 함수)
합계 → 문턱 할인 → VIP 할인

INV-1 · INV-2 · INV-3 · INV-4 · E-1 · E-2

tests/entity/

Track A

UI — Boundary

주문 품 웹페이지(Flask)
수량·VIP 입력 → 최종 금액 표시

UC-1 · UC-2 · UE-1 (+ 계측 M-1 · M-2)

tests/boundary/

상류(3장) — Mom Test 인터뷰 메모에서 제품 계약(INV/E/AC + 증거 수준)을 도출
ARRR로 구현

하류(6장) — AARRR 계측 인터페이스를 계약 ID별

ECB 방향 단속 — Boundary는 Entity를 import 할 수 있지만, Entity는 Flask를 절대 import 하지 않습니다. 이 방향이 깨지면 Layer 4 검증이 외부 의존에 오염됩니다.

파이썬 프로젝트 초기화 — 가상환경과 패키지

```
# 프로젝트 폴더 만들기
mkdir gemini-c2c-cart && cd gemini-c2c-cart
code . # VS Code 로 이 폴더 열기

# 파이썬 가상환경 생성 / 활성화
python -m venv .venv
source .venv/Scripts/activate # Windows Git Bash
# PowerShell : .venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate

# 실습 패키지 설치 및 고정
pip install flask pytest
pip freeze > requirements.txt
```

✓ 성공 신호

프롬프트 앞에 (.venv) 가 보인다
requirements.txt 가 생성됐다

⚠ 자주 하는 실수

가상환경을 켜지 않고 pip install
→ pytest 가 전역에 깔려 “왜 import 가 안 되지?”로 이
어집니다

실습 세션을 새로 열 때마다 가상환경 활성화가 첫 명령입니다 — 4장·5장 시작 전 체크포인트에 매번 등장합니다.

GitHub 저장소 연결 — 처음부터 Git을 켜다

```
git init
# GitHub 웹에서 빈 저장소(예: gemini-c2c-cart) 생성 후:
git remote add origin <저장소 주소>
git add . && git commit -m "chore: 초기 스캐폴딩"
git push -u origin main
```

커밋 메시지에 계약 ID를 적는 규율(2.8절)이 “버전 이력 = 추적성 매트릭스의 시간 축 사본”을 만듭니다.

앞으로 남길 커밋의 형식

test(entity): INV-1 ... [RED]

feat(entity): INV-1 ... [GREEN]

refactor(entity): SSOT [REFACTOR]

docs: 제품 계약 SSOT (INV-1~4)

<타입>(<계층>): <계약 ID> <무엇> [단계]

git log --oneline 만으로 “어느 계약을, 어느 단계에서 다뤘는지”가 추적됩니다 — 7.2절에서 이 이력을 다시 확인합니다.

실습 환경 최종 체크리스트 — 모두 OK면 2장으로

- ✓ 터미널에서 `gemini` 가 실행되고 Google 계정으로 로그인되어 있다
- ✓ VS Code 에서 Gemini Code Assist 채팅-Agent 탭이 열린다
- ✓ `node --version` 이 20 이상, `python --version` 이 3.11 이상을 출력한다
- ✓ `pip install flask pytest` 가 오류 없이 끝났고, 프롬프트 앞에 `(.venv)` 가 보인다
- ✓ `git status` 가 정상 동작하고, 원격 저장소가 연결돼 있다

여기까지가 실습 준비 — 이제 이 폴더 위에 2장에서 하네스를 세웁니다.

하나라도 체크되지 않으면 지금 해결하고 넘어가세요. 실습 중 환경 문제로 막히는 시간이 가장 아깝습니다.

3수준 사다리 — 프롬프트 $\subset$ 컨텍스트 $\subset$ 하네스

1

프롬프트 엔지니어링

지시문 자체 — 역할·작업·출력 형식 (개별 요청의 품질)

Gemini CLI 실체: 세션에 입력하는 한 번의 요청 · ▶ 프롬프트 박스

2

컨텍스트 엔지니어링

모델이 보는 정보 환경 — “지금 상황”을 정확히 주기

Gemini CLI 실체: @파일 참조 · GEMINI.md 계층 병합 · 테스트 실패 로그

3

하네스 엔지니어링

실행 골격 전체 — 세션을 초월해 상시 작동하는 구조물

Gemini CLI 실체: pytest 하네스 · GEMINI.md · settings.json · 커스텀 커맨드 · MCP · 금지 규칙

Agent = Model + Harness — LLM은 엔진일 뿐

컨텍스트의 왕 = 테스트 실패 로그. 실패 로그 + 최소 구현 요구 = Respond의 표준형이며, 이것이 과잉 구현을 억제합니다.

프롬프트 5트랙 — 입력 채널 매핑 (논문 표 5)

트랙	언제 쓰는가	입력 채널	프롬프트 패턴 (요지)
Discovery	계약·정책·지표 결정	gemini 세션 / Code Assist 채팅	"매트릭스로 제안하고 각 칸의 근거를 한 줄씩."
Red (Ask)	새 계약 ID 착수 (항상 먼저)	커스텀 커맨드 /tdd-red	"INV-*를 검증하는 테스트만. tests/ 만 수정."
Green (Respond)	Red 테스트 통과	gemini 세션 (Agent 자율 실행)	"[실패 로그] 최소 구현만. 구현 줄에 ID 주석."
Refactor (Refine)	전부 GREEN 이후 정리	plan 모드 리뷰 후 실행	"동작·계약 불변으로 SSOT 추출. 깨지면 멈춰."
윤영 / 그로스	지표 설계 · 퍼널 진단	gemini 세션	"최대 이탈 단계, 원인 가설, 다음 실험 한 가지."

Chat / 대화형 세션

질의 · 설계 대화 — Discovery 와 Refactor 계획

Inline Edit

단일 파일 테스트 작성 — RED 트랙의 기본 채널

Composer / Agent

다중 파일 구현과 자율 반복 — GREEN 트랙의 주력

모든 프롬프트는 이 다섯 트랙 중 하나입니다. 지금 내가 어느 트랙에 있는지 모른 채 던지는 요청이 바로 바이브 코딩입니다.

프로젝트 구조 만들기 — 폴더가 곧 계층 선언

아래 구조를 목표로 합니다.

```

gemini-c2c-cart/
├── GEMINI.md                # 하네스의 “지도” — 매 작업마다 자동 로드
├── .gemini/
│   ├── settings.json       # 승인 정책 · 도구 허용 범위 · MCP
│   └── commands/           # 커스텀 슬래시 명령 (.toml)
├── docs/PRD.md             # 제품 계약 (INV/E/AC)의 SSOT — 3장
├── pytest.ini              # 테스트 하네스 설정 (트랙 마커)
├── conftest.py             # src 임포트 경로 설정
├── src/
│   ├── cart.py             # Entity — 순수 로직 (Track B)
│   ├── app.py              # Boundary — Flask 주문 폼 (Track A)
│   └── metrics.py          # Boundary — AARRR 계측 (6장)
├── tests/
│   ├── entity/             # Track B — 불변식 (INV-*) 검증
│   ├── boundary/           # Track A — 입력·UI 계약 (E-*, UC-*)
│   └── golden/             # 회귀 기준선 — 리팩터 안전망 (4.6)

```

왜 폴더를 나누는가

테스트를 ECB 계층별로 나눠 두면 “어느 계층의 계약을 검증하는 테스트인지”가 폴더만 봐도 드러납니다.

```

mkdir -p src tests/entity \
        tests/boundary \
        .gemini/commands docs
touch src/__init__.py src/cart.py
# cart.py 는 비워 둡니다
# (RED 에서 실패를 보기 위해)

```

src/cart.py 를 일부러 비워 두는 것이 4장 RED의 출발점입니다 — 실패를 눈으로 보지 않은 테스트는 검증하는 테스트인지 알 수 없습니다.

Test Harness 구성 (pytest) — 트랙 마커 분리

```
# pytest.ini
[pytest]
testpaths = tests
python_files = test_*.py
markers =
    entity:   Track B - 불변식 (INV-*) 검증
    boundary: Track A - 입력/UI 계약 (E-*, UC-*)
    slow:     외부 호출 통합/E2E (기본 루프 제외)
    golden:   회귀 기준선 - 리팩터 전후 동작 동일성
```

```
# conftest.py
"""프로젝트 루트를 import 경로에 추가해
   `from src...` 가 동작하게 한다."""
import os, sys
sys.path.insert(0, os.path.dirname(__file__))

# ModuleNotFoundError: No module named 'src'
# → 이 파일이 루트에 없어서 생기는 1순위 오류
```

왜 마커로 나누는가

외부 의존이 생기면 slow 마커의 통합 테스트로 분리합니다. 이 분리가 Agent 가 테스트를 자동 반복 실행하는 Repeat 루프의 실용적 전제입니다.

✓ 체크포인트

- ☐ pytest 실행 시 “no tests ran” = 정상 (에러 아님)
- ☐ tests/entity/ 와 tests/boundary/ 폴더가 존재한다

테스트 하네스는 하네스의 첫 구성 요소입니다 — 외부 의존을 가짜(Fake)로 대체할 수 있게 하는 ECB 격리가 그 전제입니다.

GEMINI.md — 작업 규칙을 “지도”로

```
# Repository instructions - gemini-c2c-cart

## 구조 (ECB)
- src/cart.py      : Entity(순수 로직). Flask 를 import 하지 않는다.
- src/app.py       : Boundary(Flask). cart.py 를 import 해 재사용한다.
- tests/entity/    : INV-*      / tests/boundary/ : E-*, UC-*

## 계약 (C2C)
- 계약의 SSOT 는 docs/PRD.md 이다.
- ID 가 없는 동작은 구현하지 않는다.

## 테스트 명령
- 전체: `pytest -q` / Entity: `pytest tests/entity -q`

## 워크플로 - ARRR (RED → GREEN → REFACTOR)
- 테스트를 먼저 쓴다. “실패하는 테스트”가 항상 구현보다 앞선다.
```

금지 5조항 (Don't) — 하네스의 핵심

- ① 테스트 수정·삭제·skip 으로 우회 금지
- ② 요청하지 않은 기능 금지 (과잉 구현)
- ③ 함수 시그니처 임의 변경 금지
- ④ ID 없는 동작 구현 금지
- ⑤ 골든 마스터 파일 임의 재생성·수정 금지

작성 5원칙

- ① 짧게 — 30줄이 300줄보다 낫다
- ② 테스트 “명령”을 명시
- ③ “성격”이 아니라 “행동”
- ④ 금지 목록을 둔다
- ⑤ 자세한 건 docs/로 미룬다

규칙을 바꾼 뒤에는 /memory refresh 를 실행하거나 새 세션을 시작해야 반영됩니다. /memory show 로 병합된 컨텍스트 전체를 확인하세요.

.gemini/settings.json — 승인 정책과 도구 범위

--approval-mode	무엇	언제
plan	읽기 전용 — 분석만	Review · 스멜 탐지 (7.3)
default	모든 수정·명령을 매번 승인	기본값 (권장 시작점)
auto_edit	파일 편집만 자동 승인	GREEN 반복이 익숙해진 후
yolo	전부 자동 승인 (주의)	격리된 실습 저장소에서 만

⚠ 안전 주의

본인이 쓰지 않은 GEMINI.md · settings.json · 확장은 “신뢰할 수 없는 코드” 처럼 다루세요. git push · 배포 · rm 같은 되돌리기 어려운 명령은 default 모드에서 직접 승인합니다.

```
{
  "model": "gemini-2.5-pro",
  "context": { "fileName":
    ["GEMINI.md", "AGENTS.md"] },
  "coreTools": [
    "ReadFileTool", "WriteFileTool", "EditTool",
    "ShellTool(pytest -q)",
    "ShellTool(git status)", "ShellTool(git diff)" ],
  "excludeTools": [
    "ShellTool(git push)", "ShellTool(rm -rf)" ]
}
```

쓰기 범위는 좁게 — pytest 는 허용하고 git push 는 제외합니다. 위험 작업 전에는 gemini --checkpointing 을 켜 두면 /restore 로 복원할 수 있습니다.

커스텀 커맨드 — ARRR를 슬래시 버튼으로

/tdd-red

계약 ID의 '실패하는' 테스트만 작성. tests/ 만 수정, src/ 금지. pytest 실패 로그 전체를 보여준다. 첫 줄에 Phase: RED 선언.

/tdd-green

실패 로그를 통과시키는 '최소' 구현만. 시그니처 불변 · 구현 줄에 계약 ID 주석 · 테스트 수정/skip 금지.

/tdd-refactor

동작·계약 불변으로 구조만 정리(중복 제거-SSOT). 전후 pytest 확인. 정리할 게 없으면 "없음" — 과잉 리팩터 금지.

/golden-capture

GREEN 상태의 동작을 입력 그리드 위에 스냅샷으로 떠 회귀 기준선을 만든다. RED 상태에서는 캡처 금지 — 버그를 고정시킨다.

```
/tdd-red INV-2 (...) · /golden-capture subtotal, apply_threshold_discount · /export # .gemini/commands/*.toml - 파일명이 곧 명령 이름
```

/tdd-refactor 는 이제 골든 게이트를 포함합니다 — 리팩터 전후로 `pytest -m golden -q` 가 모두 GREEN이어야 하고, 깨지면 되돌립니다. 골든 파일 수정은 금지 5조 위반입니다.

MCP — 외부 도구 연결 (선택 심화)

```
// ~/.gemini/settings.json
{
  "mcpServers": {
    "sequential-thinking": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-sequential-thinking"]
    },
    "context7": {
      "command": "npx",
      "args": ["-y", "@upstash/context7-mcp", "--api-key", "YOUR_API_KEY"]
    },
    "github": {
      "url": "https://api.githubcopilot.com/mcp/",
      "headers": { "Authorization": "Bearer YOUR_GITHUB_PAT" }
    }
  }
}
```

context7

프롬프트 끝에 use context7 → 버전에 맞는 최신 라이브러리 문서를 찾아 옵니다. GEMINI.md 규칙으로 자동화 가능.

github

Fine-grained PAT 발급 후 등록 → “내 저장소 목록”, “이 저장소에 이슈 생성” 같은 작업이 가능해집니다.

△ MCP 서버 역시 하네스의 일부입니다 — 어떤 도구가 연결돼 있는지 /mcp 로 항상 파악하고, 신뢰할 수 없는 서버는 연결하지 않습니다. 변경 후에는 세션 재 시작 또는 /mcp refresh.

선택 심화 항목입니다 — 오늘 실습의 필수 경로는 아니지만, 실무 프로젝트에서 하네스를 확장하는 표준 통로입니다.

훅(Hooks) — 감사 로그 자동 기록

```
// .gemini/settings.json
{
  "hooks": {
    "PostToolUse": [
      { "matcher": ".*",
        "command":
          ".gemini/hooks/audit-log.sh" }
    ]
  }
}

# audit-log.sh - 실행마다 한 줄
LOG=docs/audit.log
echo "$(date) | $tool" >> "$LOG"
```

chmod +x — 실행권한 부여 (필수)

실행권한이 없으면 훅은 조용히 실패합니다. chmod +x .gemini/hooks/audit-log.sh 로 +x 를 준 뒤 ls -l 로 확인하세요 .

docs/audit.log — 감사 로그

도구 실행마다 “날짜 | 도구 | 사용자”가 한 줄씩 쌓입니다. 세션에서 파일을 수정시키면 자동으로 늘어납니다.

⚠ 실행권한(+x)이 없으면 훅은 조용히 실패합니다 — chmod +x 를 잊지 마세요. 훅은 신뢰할 수 있는 스크립트만 등록하고, 로그에 토큰·비밀번호를 남기지 않습니다.

git log가 “무엇을 커밋했는가”를 남긴다면, 감사 로그는 “에이전트가 무엇을 실행했는가”를 남깁니다 — 추적 사슬의 런타임 증거. (훅 이벤트 이름은 버전에 따라 다를 수 있음)

staging-spec 브랜치와 3종 커밋

```
# 1단계 - 계약 묶음 1개 = 브랜치 1개
git switch -c staging-spec/entity-cart

# 2단계 - RED (테스트만, 실패 확인 후 커밋)
git add tests/entity/test_cart.py
git commit -m "test(entity): INV-1 subtotal 실패 테스트 [RED]"

# 3단계 - GREEN (최소 구현만)
git add src/cart.py
git commit -m "feat(entity): INV-1 subtotal 최소 구현 [GREEN]"

# 4단계 - REFACTOR (통과 유지한 채 구조만)
git add src/cart.py
git commit -m "refactor(entity): SSOT 상수 추출 [REFACTOR]"
```

판단 기준 — 매번 이것만

- 새 계약(INV-*, E-*) 하나? → staging-spec/... 브랜치 새로
- 테스트만 바꿨나? → [RED] 커밋
- 구현만 바꿨나? → [GREEN] 커밋
- 동작 그대로, 구조만? → [REFACTOR] 커밋
- Entity+Boundary 동시 작업? → git worktree 로 폴더 분리

✓ 2장 마무리 — GEMINI.md · pytest.ini · confest.py · settings.json · commands/*.toml 존재 · /memory show 로 병합 확인 · staging-spec/entity-cart 브랜치

버전 이력 자체가 추적성 매트릭스의 시간 축 사본이 됩니다 — git log --oneline 만으로 "어느 계약을, 어느 단계에서" 다뤘는지 추적됩니다.

plan 모드로는 부족 — spec으로 역할을 나눈다

역할 분담 — 누가 무엇을 채우나

인간 (Human)

- 상위 카테고리 설정
- 진짜 상세한 spec 설계
- 전체 감독 · 검수



에이전트 (Agent)

- spec 범위 안에서
- 세부 디테일 구현
- 정해진 경계를 지킴

핵심 — “plan 모드”만으로는 부족하다. 상세한 spec이 곧 인간과 에이전트의 경계선이자 계약(INV/E/AC)이다.

내 사업 가설 — Mom Test가 겨냥할 과녁

가설 한 문장 — 주문·CS 담당자가 할인이 얹힌 주문을 확정할 때 겪는 계산 시간·계산 오류를, 계약(INV/E/AC)으로 고정된 자동 할인 계산기로 해결한다
→ 계산 시간 제거·계산 오류 0.

① 가정 1 — 문제 인식

계산을 실제 “문제”로 여기고 이미 시간을 쓴다

② 가정 2 — 경계·순서 오류

문턱·VIP 순서에서 실제 계산 오류가 난다

③ 가정 3 — 조용한 실패

잘못된 입력이 오류 없이 조용히 통과한다

나쁜 질문 (유도·미래 가정)	좋은 질문 (과거의 사실 — 가정을 캔다)
“5만원 이상이면 10% 할인하면 되죠?”	“경계 금액 근처에서 실제로 문제가 된 사례가 있었나요?”
“VIP는 추가 5% 할인하면 되죠?”	“두 할인이 동시에 적용될 때 순서 때문에 다름이 있었나요?”
“할인 후 금액은 더 작아야 하죠?”	“절대 나오면 안 되는 결과는 무엇인가요?”

산출물 — 가설 한 문장 + 위험 가정 3개. 이 둘을 쥐고 3.1 Mom Test로 넘어가면, 인터뷰가 “의견 수집”이 아니라 “계약을 캐는 Discovery”가 됩니다.

Mom Test — 계약을 발견하는 Discovery 도구

Layer 1 불변식은 어디서 오는가? 임의로 선언된 불변식은 추적 사슬의 정점이 허구라는 뜻 — 개념의 원천 자체를 검증한다 = 제품 계약(Product Contract)

① 내 아이디어를 말하지 않는다

상대의 삶·업무를 묻는다

② 과거 · 구체 · 사실만

미래 의견·가정이 아니라 실제 사건

③ 적게 말하고 많이 듣는다

칭찬·미래 약속은 무시

나쁜 질문 (유도)	좋은 질문 (과거의 사실)
"5만원 이상이면 10% 할인하면 되죠?"	"경계 금액 근처에서 실제로 문제가 된 사례가 있었나요?"
"VIP는 추가 5% 할인하면 되죠?"	"두 할인이 동시에 적용될 때 순서 때문에 다름이 있었나요?"
"할인 후 금액은 더 작아야 하죠?"	"절대 나오면 안 되는 결과는 무엇인가요?"

Mom Test는 기능 요구를 확인하는 도구가 아닙니다 — 실제 사례에서 테스트 가능한 계약을 발견하기 위한 Discovery 도구로 재정위됩니다.

계약 ID별 질문 세트 — 어떤 질문이 어떤 계약을 캐내는가

원칙을 알아도 인터뷰 자리에서는 질문이 떠오르지 않습니다. 어떤 질문이 어떤 계약을 캐내는지 미리 알고 들어가야 인터뷰가 Discovery가 됩니다.

목적	좋은 Mom Test 질문 (과거 · 구체 · 사실)
INV-1 발견	“최근 장바구니 총액을 직접 계산해야 했던 사례가 있었나요?”
INV-2 발견	“할인 기준 금액 근처에서 문제가 된 실제 사례가 있었나요?”
INV-3 발견	“일반 할인과 VIP 할인이 동시에 적용될 때 순서 때문에 다툼이나 오류가 있었나요?”
INV-4 발견	“할인 결과가 기대와 다르게 나온 적이 있나요? 어떤 결과가 절대 나오면 안 되나요?”
E-1 발견	“상품 목록이 비어 있거나 전달되지 않았을 때 실제 시스템은 어떻게 실패했나요?”
E-2 발견	“수량이나 가격이 잘못 입력된 사례가 있었나요? 그때 어떤 정보가 필요했나요?”

INV-4를 캐내는 질문(“절대 나오면 안 되는 결과는?”)이 특히 중요합니다 — AC에는 절대 나타나지 않는, 가장 높은 수준의 불변식이 여기서 나옵니다.

AC · Invariant · Error Contract — 구분과 승격

구분	의미	예
AC (수용 기준)	사용자가 받아들일 조건 — G-W-T	"5만원 이상이면 할인된다"
Invariant (INV)	어떤 입력에서도 깨지면 안 되는 규칙	"할인 후 금액이 원금보다 커질 수 없다"
Error Contract (E)	잘못된 입력에 대한 명시적 오류 규칙	"qty가 음수이면 ValueError"

AC → INV 승격의 예

AC-1 "5만원 이상 10% 할인"에서 INV-2가 바로 나오지만,
 Mom Test로 "절대 발생하면 안 되는 결과는?"을 물어야
 → INV-4: $0 \leq \text{final_total} \leq \text{subtotal}$ 이라는 더 높은 수준의 불변식이 나옵니다.
 AC의 Given-When-Then은 Ask 단계 테스트의 기계적 번역 원본입니다.

증거 수준 — 희망과 증거를 섞지 않기

L0

협상 불가 외부 제약 / 과거 행동 + 수치 동반

L1

고객 발화·실측으로 뒷받침되는 항목

L2

근거 약한 추정 — 우선순위 하락·추가 검증

이 표기가 희망 사항과 증거의 혼재를 구조적으로 차단합니다 — L2는 우선순위에서 밀리거나 추가 검증 대상이 됩니다.

AC에서 계약을 뽑아내는 4가지 규칙

①

AC 그 자체

→ 테스트 조건

Given-When-Then을 그대로 테스트로 번역

②

AC 안에 숨은 항상성

→ Invariant (INV-*)

어떤 입력에서도 깨지면 안 되는 규칙

③

AC 밖의 잘못된 입력

→ Error Contract (E-*)

명시적 오류 규칙 — 조용한 실패 금지

④

AC에 없는 기능

→ Out of Scope (OOS-*)

만들지 않는다 — 과잉 구현 차단

예) AC-1 “5만원 이상이면 10% 할인한다”

- ① 테스트 조건 → 50000 이면 적용 / 49999 이면 미적용 (경계 2행)
- ② 여기서 바로 나오는 불변식 → INV-2 : $\text{amount} \geq 50000 \rightarrow \text{round}(\text{amount} \times 0.9)$

그런데 AC만으로는 부족하다 — Mom Test 한 걸음 더

- “할인 계산에서 절대 발생하면 안 되는 결과는?”
- “할인했는데 원래보다 비싸지면 안 됩니다.”
- INV-4 : $0 \leq \text{final_total} \leq \text{subtotal}$ ← AC에는 없던 불변식

AC는 Invariant의 원천이 될 수 있지만, Invariant는 AC보다 강한 도메인 계약입니다. 그 간극을 메우는 것이 Mom Test입니다.

실습 ① — 인터뷰 진행하기 (Mom Test 실전)

3.4절의 [인터뷰 메모]는 완성된 상태로 주어집니다. 그런데 실무에서는 그 메모를 스스로 만들어야 합니다 — Gemini에게 ① 질문지를 짜게 하고, ② 고객 역할을 맡겨 인터뷰를 연습합니다.

▶ ① 인터뷰 가이드 생성

당신은 Mom Test 규칙을 엄격히 적용하는 인터뷰 설계자다.
가설 job: "주문 담당자가 할인 금액을 오류 없이 확정한다"

1. 과거 행동을 캐는 질문 8개 (미래·의견 금지)
2. 각 질문 옆에 "캐내려는 계약 후보" 한 줄
3. 반드시 포함:
 - "절대 발생하면 안 되는 결과는?" → INV 후보
 - "잘못 입력했을 때 뭐가 일어났나?" → E 후보
 - "그때 수치가 어떻게 됐나?" → L0/L1 판정
4. 물으면 안 되는 유도 질문 3개를 반례로

▶ ② 역할극 인터뷰 — Gemini가 고객이 된다

지금부터 당신은 온라인 쇼핑물 주문 담당자다. 나는 인터뷰어다.

- 과거에 실제로 겪은 일만 말한다. 해결책을 제안하지 않는다.
- 내가 유도 질문("~하면 되죠?")을 하면, 답하기 전에 먼저 "△ 유도 질문입니다" 라고 지적하고 어떻게 바꿀지 알려 준다.
- "인터뷰 종료"라고 하면 [인터뷰 메모] 형식으로 정리한다. 없는 사실을 지어내지 않는다.

첫 질문을 기다린다.

△ 역할극에서 Gemini가 술술 답한다고 좋아하지 마세요 — 유도 질문에 단순히 답하면 잘못 세팅된 것입니다. ②의 출력이 곧 3.4절의 입력 형식이므로, 역할극 → 메모 → 계약표 → PRD.md 가 한 줄로 이어집니다.

실습 ② — 인터뷰 메모를 계약으로 변환

[인터뷰 메모] — 실습 입력

- A 3개(12,000) + B 1개(30,000) = 66,000. 5만 원 넘으면 10% 할인. 기대 59,400. 엑셀 계산 10분
- VIP는 문턱 할인 후 추가 5%. 순서 문턱→VIP 고정 (순서 뒤바뀜 클레임 1건)
- 수량 -1 입력 시 합계 0원 표시(기대: 오류) / items 없이 제출 시 500 에러
- 쿠폰·배송비: 언급 없음 / "50,000원 정확히" 포함 여부 미확인

⚠ 자주 하는 실수

"50,000원 정확히"처럼 미확인 경계는 L2로 분류돼야 정상입니다. 쿠폰·배송비가 계약표에 나타나면 과잉 명세 — OOS로 밀려 있는지 확인하세요.

▶ Gemini CLI — Discovery 트랙

당신은 Mom Test 방식으로 도메인 규칙을 발견하는 Product Discovery / TDD 분석 에이전트입니다.

- Step 1. 각 항목을 증거 수준 (L0/L1/L2)으로 분류
- Step 2. Invariant / Error / AC / Out-of-Scope 추출
- Step 3. ID 부여: INV-n, E-n, AC-n, OOS-n
- Step 4. 테스트 가능한 문장으로 변환
나쁜 예: "할인이 잘 적용되어야 한다"
좋은 예: "amount >= 50000 이면 round(amount*0.9)"
- Step 5. AC 는 Given-When-Then 으로
- Step 6. 출력: |ID|계약|증거수준|계층|테스트 아이디어|

규칙: 근거 없는 동작을 계약 ID 로 만들지 마라.

발화가 계약이 되는 지점입니다 — 이 변환이 정확해야 아래로 이어지는 테스트·구현·계측 전체가 근거를 갖습니다.

실습 결과 통일 — 확정 계약표를 docs/PRD.md 에 고정

ID	계약 (불변식 / 에러)	증거 수준	계층
INV-1	$\text{subtotal}(\text{items}) == \sum(\text{price} \times \text{qty})$	— (수학적 정의)	Entity
INV-2	$\text{amount} \geq 50000 \rightarrow \text{round}(\text{amount} \times 0.9)$ / 미만은 그대로 (경계 포함)	L1	Entity
INV-3	문턱 할인 적용 후 VIP면 $\text{round}(\times 0.95)$. 순서 문턱→VIP 고정	L1	Entity
INV-4	모든 입력에서 $0 \leq \text{final_total} \leq \text{subtotal}$	L0	Entity
E-1	items is None $\rightarrow$ TypeError	L0	Boundary*
E-2	price 또는 qty 가 음수 $\rightarrow$ ValueError (인덱스 포함)	L0	Boundary*

▶ docs/PRD.md 를 생성하라. 계약표 6행 + AC (G-W-T) + 구현 금지 (OOS: 쿠폰·세금·배송비·포인트) 를 포함한다. 아직 구현 완료라고 쓰지 마라 — 현재 단계는 RED 준비다.

* E-1·E-2 는 본래 Boundary 관심사(입력 검증)지만, 이 작은 예제에서는 Entity 함수 입구에서 막고 tests/boundary/ 로 계약을 검증합니다.

Track B — Discovery ①②③: 목표 · 실패 조건 · 계약

① 목표 좌표 — 무엇을 보장하는가

보장 합계 → 문턱 할인 → VIP 할인을 정확·일관되게 계산
ECB Entity (순수 로직 · 외부 I/O·UI 문자열 없음)
C2C Layer 4 구현 단위 + Layer 1 불변식
트랙 Track B — Domain/Logic (tests/entity/)

```
subtotal(items)
apply_threshold_discount(amount)
final_total(items, is_vip=False)
# 대상 함수 3개 - 시그니처는 여기서 고정된다
```

② PRD — “무엇이 실패인가” (L0~L3 사다리)

L0 · 계약 위반

items 가 None인데 조용히 0을 반환 → 예외를 던져야 (E-1)

L1 · 경계

합계가 정확히 50,000인데 할인이 안 된다 → “이상(≥)” 위반 (INV-2)

L2 · 도메인 규칙

VIP인데 5%가 빠지거나 문턱→VIP 순서가 뒤바뀐다 (INV-3)

L3 · 수학적 불변

최종 금액이 subtotal보다 커진다 → 할인이 금액을 늘렸다 (INV-4)

③ 계약 확인 — 이 모듈이 충족할 계약은 3.4절에서 확정된 INV-1~4 · E-1~2. 계약은 docs/PRD.md 한 곳에만 존재하고(SSOT), 테스트·구현·커밋은 ID로만 참조합니다.

테스트 플랜 — RED 전에 테스트 이름을 못 박는다

이 표가 없으면 ⑤ Ask(RED)에서 AI가 테스트 이름을 즉흥 작명하게 되고, 4.8절의 추적성 매트릭스 대조가 사후 짜맞추기로 전락합니다. 이것이 매트릭스의 좌측 3열입니다.

계약 ID	테스트 이름	파일	유형	케이스 (입력 → 기대)
INV-1	test_subtotal_sums_price_times_qty	tests/entity/	예제	$1000 \times 3 + 2000 \times 2 \rightarrow 7000$
INV-2	test_threshold_applies_at_50000	tests/entity/	경계	$50000 \rightarrow 45000$ (경계 포함)
INV-2	test_threshold_not_applied_below	tests/entity/	경계	$49999 \rightarrow 49999$ (할인 없음)
INV-3	test_vip_order_and_value	tests/entity/	순서	$60000 + \text{VIP} \rightarrow 54000 \rightarrow 51300$
INV-4	test_discount_never_increases_amount	tests/entity/	속성	빈 카트·49999·50000·VIP 전부 $0 \leq \text{final} \leq \text{subtotal}$
E-1	test_none_items_raises_type_error	tests/boundary/	예외	<code>None</code> → <code>TypeError</code>
E-2	test_negative_qty_reports_index	tests/boundary/	예외	<code>qty=-1</code> → <code>ValueError</code> (인덱스 포함)

5종 유형 — 예제(정의) · 경계(임계값 양쪽을 반드시 2행으로) · 순서(적용 순서가 결과를 바꾼다) · 속성(모든 입력에서 성립 — 단일 예제 금지) · 예외(Error Contract는 항상 이 유형)

④ 스켈레톤 — 시그니처 + TODO (완성 코드는 감춤)

```
"""장바구니 할인 계산 - Entity 계층 (Track B / Domain).

계약 (Contracts): docs/PRD.md 참조 (SSOT)
INV-1 subtotal == E(price*qty)
INV-2 amount>=50000 → round(*0.9) (경계 포함)
INV-3 문턱할인 후 VIP면 round(*0.95). 순서 고정
INV-4 0 <= final_total <= subtotal
E-1 items is None → TypeError
E-2 price/qty 음수 → ValueError(인덱스 포함)
"""
THRESHOLD = 50000          # INV-2 문턱 금액 (SSOT)

def subtotal(items):
    # TODO(E-1): items is None 이면 TypeError
    # TODO(E-2): 음수면 ValueError(인덱스 포함)
    # TODO(INV-1): E price*qty 반환
    raise NotImplementedError
```

스켈레톤 프롬프트 5원칙

- ① 단계 한 줄 선언 [STRUCTURAL]
- ② 수정 허용 파일 명시
- ③ 계약 ID 표 제시
- ④ ECB 한 줄 (import 방향)
- ⑤ 완료 기준 (import 가능 · NotImplementedError)

금지 — 여기서 구현하지 않는다

실행 가능한 할인 로직을 쓰지 않는다. 테스트 파일도 만들지 않는다. “무엇을(계약) / 어디까지(구조만) / 무엇을 하지 말지(과잉 금지)”를 분리해 지시합니다.

완성 코드를 감추는 것은 교육 장치이자 추적성 장치입니다 — 학습자와 AI 모두 “계약 → 테스트 → 구현 줄” 순서로만 코드에 도달하게 됩니다.

Loop 1 — subtotal (INV-1 · E-1 · E-2)

A = Ask (RED)

/tdd-red INV-1, E-1, E-2

```
# tests/entity/test_cart.py
def test_subtotal_sums_
    price_times_qty(): # INV-1
    items=[{price:1000,qty:3},
            {price:2000,qty:2}]
    assert subtotal(items)==7000

# tests/boundary/test_app.py
# E-1 TypeError / E-2 ValueError
```

\$ pytest -q → 3 failed
NotImplementedError (의도된 RED)

R = Respond (GREEN)

/tdd-green INV-1, E-1, E-2

```
def subtotal(items):
    if items is None: # E-1
        raise TypeError(...)
    for i, it in enumerate(items):
        if it[price]<0 or it[qty]<0:
            raise ValueError( # E-2
                f"... at index {i}")
    return sum(it[price]*it[qty]
               for it in items) # INV-1
```

실패 로그가 곧 Context — 구현 줄마다 계약 ID 주석(추적의 못)

R = Refine (REFACTOR)

권장 워크플로 — 턴을 나눈다

- ① 스멜 탐지만 (plan 모드, 수정 없음)
- ② 리팩터 계획 확정
- ③ 실행 1개만 → pytest → refactor 커밋

“정리할 게 없음을 확인하는 것”도 정식 단계입니다.
이번 Loop는 스킵하고 Loop 2에서 실제 리팩터를 수행합니다.

✓ Loop 1 체크포인트 — pytest -q 가 3 passed · 구현 줄에 INV-1-E-1-E-2 주석 · test(RED) → feat(GREEN) 커밋이 계약 ID와 함께 분리되어 있다

Loop 2 — apply_threshold_discount (INV-2)

```
/tdd-red INV-2
- apply_threshold_discount(50000) == 45000 # 경계 포함
- apply_threshold_discount(49999) == 49999 # 할인 없음
# → 2 failed (의도된 RED)
```

```
/tdd-refactor → 매직 넘버를 SSOT 상수로

THRESHOLD      = 50000 # INV-2 문턱 금액 (SSOT)
THRESHOLD_RATE = 0.9   # INV-2 문턱 할인율 (SSOT)

def apply_threshold_discount(amount):
    if amount >= THRESHOLD: # INV-2 (경계 포함)
        return round(amount * THRESHOLD_RATE)
    return amount
```

⚠ 자주 하는 실수 — 경계 우회

amount > 50000 으로 짜면 50000 테스트가 실패합니다. 요구는 “이상(≥)”입니다.

에이전트가 테스트를 바꿔 통과시키려 하면 즉시 막으세요 — GEMINI.md 금지 1조(테스트 약화) 위반이며, 백이 꼽은 “테스트를 속이는 정황”입니다.

3중 커밋 — 각 단계 직후 순서대로

```
test(entity): INV-2 경계 실패 테스트 [RED]
feat(entity): INV-2 문턱 할인 구현 [GREEN]
refactor(entity): THRESHOLD_RATE SSOT 추출 [REFACTOR]
```

0.9 가 매직 넘버 냄새를 풍기기 시작하는 지점 — 의미를 드러내는 상수로 추출하면 INV-2의 SSOT가 됩니다. 동작 불변이므로 refactor 커밋으로 분리합니다.

리팩터 전 — 골든 마스터로 회귀 기준선을 뜯다

Tidy First 3규칙 ③ — “구조 변경 전후로 동작이 변하지 않았음을 기계적으로 확인한다”. 그런데 `pytest -q` 는 “내가 단언한 몇 개 케이스가 안 깨졌다”만 증명하지, “입력 공간 전체에서 동작이 동일하다”는 증명하지 않습니다.

```
/golden-capture apply_threshold_discount

# tests/golden/cart_golden.json
{ "apply_threshold_discount": [
  {"in": 49999, "out": 49999},
  {"in": 50000, "out": 45000},
  {"in": 60000, "out": 54000},
  {"in": 66000, "out": 59400} ] }

@pytest.mark.golden
def test_golden_master_unchanged():
    for c in GOLDEN["apply_threshold_discount"]:
        assert apply_threshold_discount(c["in"]) == c["out"]
```

규율 ① — GREEN 상태에서만 캡처

RED 상태에서 뜨면 버그를 기준으로 고정시킵니다. 반드시 `pytest -q` 가 통과한 뒤에 캡처합니다.

규율 ② — 골든 파일을 고쳐 통과시키지 않는다

리팩터 후 골든이 깨지자 골든을 재생성해 통과시키는 것은 “테스트를 바꿔 통과시키기”와 같은 회피입니다 → GEMINI.md 금지 5조.

판정 기준 — 골든 diff가 0이 아닌 refactor 커밋은 그 자체로 규칙 위반입니다. 계약이 실제로 늘었다면 `chore(golden): ...` 커밋으로 의도적 재생성을 이력에 남깁니다.

Loop 3 — final_total (INV-3 순서 · INV-4 속성)

```
/tdd-red INV-3, INV-4

def test_vip_order_and_value():          # INV-3
    items = [{"price": 60000, "qty": 1}]
    assert final_total(items, is_vip=True) == 51300
    # 60000 → 54000(문턱) → 51300(VIP) - 순서 검증

def test_discount_never_increases_amount(): # INV-4
    for items, vip in samples:           # 빈 카트·49999·50000·VIP
        assert 0 <= final_total(items,vip) <= subtotal(items)
```

재사용이 곧 보장 — 이미 만든 subtotal.apply_threshold_discount 를 재사용하라고 명시하면 중복 없이 통과하고, 재사용 자체가 INV-4(할인만 적용 → 금액이 늘 수 없음)를 자연스럽게 보장합니다.

```
/tdd-green INV-3, INV-4

def final_total(items, is_vip=False):
    amount = apply_threshold_discount(
        subtotal(items))          # INV-3 문턱 먼저

    if is_vip:
        amount = round(amount * VIP_RATE) # INV-3 그다음
    return amount                 # INV-4 할인만 적용 → subtotal 이하
```

INV-4 는 여러 입력을 도는 “속성 테스트”로 둡니다 — 단일 예제가 아니라 성질 자체를 검증하는 것이 Layer 1 불변식의 본래 형태입니다.

0.95 도 VIP_RATE 상수로 추출해 INV-2의 상수들과 일관되게 둡니다. 정리할 게 없다면 억지로 만들지 않습니다 — 과잉 리팩터 금지.

완성 코드와 C2C 추적성 매트릭스

```
THRESHOLD      = 50000    # INV-2 문턱 금액 (SSOT)
THRESHOLD_RATE = 0.9      # INV-2 문턱 할인율 (SSOT)
VIP_RATE       = 0.95     # INV-3 VIP 추가 할인율 (SSOT)

def subtotal(items):
    if items is None:      # E-1
        raise TypeError("items must not be None")
    for i, it in enumerate(items): # E-2
        if it["price"] < 0 or it["qty"] < 0:
            raise ValueError(f"... at index {i}")
    return sum(it["price"] * it["qty"] # INV-1
               for it in items)

def final_total(items, is_vip=False):
    amount = apply_threshold_discount(subtotal(items)) # INV-3
    if is_vip: amount = round(amount * VIP_RATE)      # INV-3
    return amount                                     # INV-4
```

```
$ pytest -q
6 passed in 0.07s ✓
```

▶ 매트릭스를 Gemini에게 생성·검사시킨다

docs/PRD.md 의 계약표와 tests/src/ 를 대조해 추적성 매트릭스를 만들어라.

그리고 별도로 보고하라:

- ① 빈칸이 있는가 (계약은 있는데 테스트/구현이 없는 행)
- ② 계약 ID 주석이 없는 “정체불명” 코드 줄

파일은 수정하지 마라. docs/traceability.md 로 저장만.

모든 구현 줄에 계약 ID가 달려 있어 “이 줄이 어느 계약을 지키는가”가 코드만 봐도 읽힙니다 — 표를 수작업으로 채우지 말고 Gemini에게 검사시키는 것이 Review 트랙의 표준입니다.

Track A — 목표 · UI 계약 · 테스트 플랜

① 목표 좌표

보장 주문 품에서 수량·VIP를 입력받아 최종 금액을 정확히 표시
 ECB Boundary (입력 검증·출력 형식화. 계산은 Entity에 위임)
 C2C Layer 2 UI Contract Test
 대상 Flask app: GET / (품) · POST /calc (결과)

ID	계약	실패 시나리오
UC-1	GET / 는 200 + 수량·VIP 품 포함	L1 품이 안 됨
UC-2	POST /calc → final_total 결과 표시	L2 금액 오표시
UE-1	qty 가 숫자가 아니면 400	L0 서버 오류

테스트 플랜 — UC-1 test_form_renders (예제) · UC-2 test_calc_shows_final_total (예제) · UE-1 test_calc_rejects_non_numeric_qty (예외)

ECB 방향 단속 — 이 장의 핵심

Boundary → Entity ✓ 허용
 from src.cart import final_total

Entity → Boundary ✗ 절대 금지
 cart.py 는 Flask 를 import 하지 않는다

왜 51300 을 다시 계산하지 않는가

계산 자체는 Entity의 final_total 이 이미 INV-1~4로 보장합니다. Boundary 테스트는 “계산이 맞나”가 아니라 “올바른 함수를 부르고 결과를 형식에 맞게 돌려주나”를 검증합니다.

Entity가 Boundary를 import 하는 순간 Layer 4의 검증이 외부 의존에 오염되어 4계층 구조가 무너집니다 — 이 방향 단속은 GEMINI.md로 상시 강제됩니다.

④ 스켈레톤 — 좋은 프롬프트와 나쁜 프롬프트

[STRUCTURAL] Boundary 스캐폴딩. 동작 구현 (GREEN)은 하지 않는다.

```
## 수정 허용: src/app.py 만. tests/ 와 src/cart.py 는 안 건드린다.
## 계약 (docs/PRD.md): UC-1 · UC-2 · UE-1
## ECB: from src.cart import final total 만 허용 (Boundary→Entity).
    할인 공식 · 문턱 · VIP 순서를 재구현하지 않는다.
## 산출물: app = Flask(__name__), @app.get("/"), @app.post("/calc"),
    핸들러 본문은 raise NotImplementedError, TODO 주석에 계약 ID.
    폼 필드 SSOT: price · qty · vip(checkbox), action=/calc
## 금지: 구현 로직, HTML 상세, DB, 세션, 쿠폰, 추가 라우트.
## 완료 기준: import 가능, 핸들러 호출 시 NotImplementedError.
```

나쁜 프롬프트 — 왜 나쁜가

“Flask 주문 폼을 완성해줘”

UC/UE 범위·필드명·응답 형식이 빠짐

“할인 로직도 테스트해줘”

INV는 Entity 책임 — 역할 침범

“예쁜 UI로 만들어줘”

ID 없는 동작 = 과잉 구현

RED와 GREEN을 한 프롬프트에

GEMINI.md의 RED→GREEN 순서 위반

“음수 qty도 400으로”

UE-1은 비슷자만. 음수는 E-2(Entity)

스켈레톤 프롬프트에는 “단계 한 줄 · 수정 허용 파일 · 계약 ID 표 · ECB 한 줄 · 완료 기준” 다섯 가지를 반드시 넣습니다.

ARRR — Flask test_client 로 RED → GREEN → REFACTOR

```
# tests/boundary/test_app.py (RED)
from src.app import app

def test_form_renders():                # UC-1
    res = client().get("/")
    assert res.status_code == 200
    assert b'name="qty"' in res.data

def test_calc_shows_final_total():      # UC-2
    res = client().post("/calc",
        data={"price":60000,"qty":1,"vip":"on"})
    assert b"51300" in res.data

def test_calc_rejects_non_numeric_qty(): # UE-1
    assert client().post("/calc",
        data={"qty":"abc"}).status_code == 400
```

```
# src/app.py (GREEN)
from flask import Flask, request
from src.cart import final_total      # Boundary → Entity

app = Flask(__name__)

@app.get("/")
def form():                          # UC-1
    return '<form method="post" action="/calc"> ... </form>'

@app.post("/calc")
def calc():
    try:                              # UE-1
        price = int(request.form["price"])
        qty = int(request.form["qty"])
    except (KeyError, ValueError):
        return "수량/가격은 숫자여야 합니다", 400
    total = final_total(...)          # UC-2 (재사용)
```

REFACTOR — 품 HTML이 길어지면 템플릿 파일로 분리(Extract)하는 것이 자연스러운 다음 정리입니다. 단 동작·계약 불변을 유지하고 refactor 커밋으로 분리합니다. 이번 실습 범위에서는 인라인 문자열을 유지해도 됩니다 (과잉 리팩터 금지).

브라우저 없이도 HTTP 동작을 검증할 수 있는 test_client 가 Boundary 트랙의 하네스입니다 — 외부 의존을 경계 뒤에 격리한 덕분입니다.

브라우저로 시각 확인 — 사람이 눈으로 보는 계약

```
flask --app src/app run
# http://127.0.0.1:5000 접속 → 값을 바꿔 가며 확인
```

가격	수량	VIP	기대
60000	1	☒	51300
60000	1	☐	54000
50000	1	☐	45000
49999	1	☐	49999
60000	abc	☐	400 에러

왜 눈으로도 보는가

test_client 가 계약을 보장하지만, 계약이 "사용자가 실제로 겪는 것"과 일치하는지는 사람이 한 번 봐야 합니다. 이 확인이 Layer 2 UI 계약의 마지막 검증입니다.

✓ 5장 마무리 체크포인트

- ☐ pytest -q 가 9 passed (4장 6 + 5장 3)
- ☐ src/cart.py 에 from flask 가 없다
- ☐ app.py 가 final_total 을 재사용한다 (재계산 금지)
- ☐ 브라우저에서 품이 뜨고 금액이 바뀐다

경계값 50000 → 45000 이 나오는지 반드시 눈으로 확인하세요 — INV-2의 "이상(≥)"이 코드에서 "초과(>)"로 미끄러지는 사고가 가장 흔합니다.

하류 귀환 — AARRR 재정의와 역산 설계

△ 용어 주의 — 네 글자 AARRR(Ask-Respond-Refine-Repeat) = Dual-Track TDD의 실행 리듬 / 다섯 글자 AARRR(Acquisition-Activation-Retention-Referral-Revenue) = 그로스 퍼널 지표. 별개의 개념입니다.

① 재정의 — 제품 맥락으로 조작적 정의

결제가 없는 실습 앱에서 Revenue 는 '주문 완료'로 재정의됩니다.

논문 사례에서도 동일합니다 —

- 사내 RAG 도구: Revenue = 비용 절감 · 안전 리스크 저감
- 소비자 MVP: Revenue = 가입 전환

② 역산 설계 — 목표에서 거꾸로

최종 목표: 30일 내 주문 완료 300건

가정: 방문 → 계산 시도(Activation) 70%

계산 → 주문(Revenue) 20%

→ 필요 Activation $\approx 1,500$ / 필요 방문 $\approx 2,143$

가정은 반드시 명시적으로 기록하고, 실측 후 갱신합니다 (docs/aarr.md).

추적성 매트릭스가 채워졌다는 것은 코드가 계약을 지킨다는 뜻이지, 계약이 옳았다는 뜻은 아닙니다 — 그것을 판정하는 것이 하류 귀환입니다.

계측 코드화 — 계측 모듈도 ARRR로 만든다

ID	계측 계약	계측
M-1	POST /calc 성공 시 calc_completed 이벤트 정확히 1회	Boundary
M-2	400 거절 시 calc_rejected 기록 (성공 이벤트 없음)	Boundary

```
# src/metrics.py - Boundary 계측 (ARRR 계측)
_collector = [] # 실전에서는 분석 SDK 로 교체

def record(event: str): # M-1, M-2
    _collector.append(event)

# src/app.py 의 calc() 내부에 두 줄:
# 성공 return 직전: record("calc_completed") # M-1
# 400 return 직전: record("calc_rejected") # M-2
```

계측도 계약이다 — RED 부터

실제 분석 SDK 대신 주입 가능한 가짜 수집기(하네스)로 검증합니다. 성공 기준 자체가 관측 가능한 코드 산출물이 되고, 계약과 지표가 연결됩니다.

폐회로 — 발화가 계약이 되고, 계약이 코드가 되고, 코드가 질문으로 돌아온다

계약(개념) → 테스트 → 구현 → 계측 → 실측

수치가 목표에 미달하면 그 수치가 다음 회전의 Discovery 질문("어느 단계에서 왜 이탈하는가")을 생성합니다.

✓ 6장 마무리 — docs/aarr.md 에 재정의·역산·가정 기록 · pytest -q 전부 GREEN (11 passed) · 계측 이벤트명이 문서의 이벤트명과 일치 (명명 규약 = 추적 수단)

ARRR 반복과 “확장 ↔ 수축” 균형

	확장 (Expansion)	수축 (Contraction)
무엇	새 계약 ID → 새 RED → GREEN	중복 제거 · SSOT 추출 · 이름 정리
커밋	test / feat	refactor
신호	기능이 늘어난다	코드가 줄거나 명료해진다 (동작 불변)

리듬 — 2~3개 확장마다 수축을 끼운다

예: INV-5 “수량 0인 항목은 합계에서 제외”를 추가할 때마다 R=Repeat 로 RED→GREEN→REFACTOR를 한 바퀴 돌립니다.

기능 추가(확장)만 반복하지 않는 것 — 이것이 “씨앗을 먹지 않는” 리듬입니다.

⚠ 결길 신호를 감시

Agent 가 요청하지 않은 계약(기능)을 추가하면
→ 멈추고 되돌린다 (과잉 구현)

Agent 가 테스트를 바꾸거나 지워 통과시키려 하면
→ 즉시 거절 (테스트 약화)

중간 결과를 흘려보지 말고, “다음엔 INV-5만”처럼 한 계약씩 좁혀 지시한다

Augmented Coding 의 실천 — 개발자가 주도권을 쥐고 중간 결과를 감시하며, 필요하면 개입·중단합니다.

커밋 추적과 Review — 수정 없이 점검 (plan 모드)

```
$ git log --oneline
alb2c3d refactor(entity): VIP_RATE SSOT 추출 [REFACTOR]
9f8e7d6 feat(entity): INV-3.4 final total [GREEN]
7c6b5a4 test(entity): INV-3.4 실패 테스트 [RED]
5a4b3c2 refactor(entity): THRESHOLD_RATE 추출 [REFACTOR]
3b2a1c0 feat(entity): INV-2 문턱 할인 구현 [GREEN]
1a0b9c8 test(entity): INV-2 실패 테스트 [RED]
```

이력 자체가 추적성 매트릭스의 시간 축 사본

형식: <타입>(<계층>): <계약 ID> <무엇> [단계]
git log 만으로 “어느 계약을, 어느 단계에서” 다뤘는지 추적됩니다.

Review — gemini --approval-mode plan

코드를 고치지 않는 읽기 전용 세션에서 5항을 점검:

- 구현 줄의 계약 ID 주석 누락을 지목하라
- 추적성 매트릭스(계약↔테스트↔구현)의 빈칸
- 과잉 구현 · 우회(assert True / skip / 테스트 변경)
- 구조 변경과 동작 변경이 한 커밋에 섞였는가 (Tidy First)
- Entity 가 Boundary 를 import 하지 않는가 (ECB 방향)

발견 사항만 목록으로 보고하고, 수정 제안은 별도로 적어라. 파일은 고치지 마라.

머지 전 감리 — 사람이 규칙을 외워 검사하는 것이 아니라, 규칙을 프롬프트로 고정해 AI에게 검사시키는 것이 Review 트랙입니다.

문서화 · 골든셋 평가 · KPT 회고

/export — 세션마다 추적 기록을 남긴다

Report/NN.REPORT.md 세션 요약 (결정·산출물·다음 단계)
Prompting/NN.Transcript.md 턴별 대화 전문 + 변경 파일 표

번호가 세션마다 1씩 증가하며, 이 문서 이력이 “개념→코드→결과”의 추적 기록을 문서로 담습니다.

골든셋 평가 (G-1~G-5) — 계약이 옳았는가

G-1 정상 66,000 → 59,400 · G-2 경계 50,000 할인 적용 · G-3 순서 VIP 51,300

G-4 방어 qty=-1 → ValueError · G-5 불변 final ≤ subtotal

골든 마스터는 “동작이 변했는가”를, 골든셋은 “계약이 옳았는가”를 묻습니다.

✓ 과정 전체 마무리 체크포인트

- ✓ Entity(4장) · Boundary(5장) · 계층(6장)을 모두 8단계 골격으로 완성했다
- ✓ pytest -q 전부 GREEN, 추적성 매트릭스가 채워진다
- ✓ RED · GREEN · REFACTOR 커밋이 계약 ID와 함께 분리돼 있다
- ✓ 확장 뒤 수축(리팩터)으로 균형을 맞췄다
- ✓ 계약 ID 없는 “정체불명” 코드 줄이 없다
- ✓ /export 로 세션 기록을 남겼다

이 장의 활동이 논문 STEP 13(골든셋 평가 · AARRR 실측 · 추적성 매트릭스 점검 · KPT 회고)에 해당합니다.

제조 모터 과열 트러블슈팅 RAG 어시스턴트 (안전 중심)

Mom Test 가 드러낸 세 가지 고통

- P1 경보 시 여러 PDF를 뒤지느라 초기 대응 10~20분 지연
- P2 근거 없는 경험칙 추정에 따른 오진 위험
- P3 급박한 상황에서 LOTO 확인이 뒤로 밀리는 안전 절차 누락

환각 함정 골든셋

“문서에 없어서 확신이 없었던 질문”을 별도로 수집 — RAG가 반드시 답변을 거절해야 하는 질문 유형의 원천이 됩니다.

ID	계약 (대부분 L0)
INV-2	답변은 반드시 매뉴얼 근거 id / 섹션을 포함
INV-3	원인 / 점검 / 안전 / 근거 4분할 형식
INV-4	근거 부재 시 ‘문서상 확인 불가’ 강제
INV-5	역할별 열람 통제 (operator 원문 마스킹)
E-1	근거 없는 수치 단정 금지
E-2	안전 조치(LOTO·보호구) 없는 정비 지시 금지
E-4	실제 설비 제어 기능의 MVP 포함 금지

종단 추적 — 발화 “급하면 LOTO 확인을 나중에 한다” → 고통 P3 → E-2·AC-3 계약화 → 프롬프트 하네스 검사 테스트 + 골든 퀘스천 E2E → 하네스 구현 → 안전 문구 포함률 계측으로 귀환.

소비자 모바일 그로스 MVP (성장 중심)

정량 목표에서 출발 — 유입 3,000 · 가입 300 (30일)

job: “30초 안에 나만의 재미있는 결과물을 만들어 SNS에 공유하고 반응을 얻고 싶다”

→ 5개 구현 후보를 ICE로 비교 → AI 스타일 변환 앱 선정

AARRR 역산

가입 300 → Activation 대비 가입 전환율 약 14.3%, 유입 대비 Activation 70% 가정 → 상위 퍼널 규모 도출. K-factor 0.3 으로 바이럴 루프 설계.

ID	계약 — 성장과 신뢰가 같은 형식으로
INV-1	60초 TTV — 최초 실행 후 60초 내 결과 확인
INV-2	공유 이미지에 앱 설치 딥링크 자동 포함
INV-3	게스트 1회 무료 · 가입 시 워터마크 제거
INV-4	업로드 사진의 24시간 내 자동 삭제 (프라이버시)
INV-5	스토어 심사 준수 (규제 계약)
E-1	변환 실패 시 크레딧을 차감하면 안 된다
E-2	부적절 콘텐츠 처리 금지

종단 추적 — 발화 “가입하라고 해서 그냥 달았다” → 고통 P2 → INV-3-E-5-AC-3 → AccessGate 매트릭스 테스트 → 가입 전환 계측으로 귀환. 성장 계약과 규제 계약이 같은 형식으로 공존합니다.

같은 C2C 구조, 도메인별 인스턴스화

구성 요소	사례 1 — 모터 과열 RAG (안전)	사례 2 — 그로스 MVP (성장)
Layer 1 불변식	근거 인용 강제 · 확인 불가 응답 · 안전 조치 포함	60초 TTV · 공유 딥링크 · 24시간 사진 삭제
Layer 2 UI 계약	역할별 원문 마스킹 · 4분할 답변 양식	크레딧 소진 시 가입 유도 · 실패 시 재시도
Layer 3~4	검색기 · 파이프라인 · 파서 (환각 방어)	크레딧 · 공유 · 변환 유스케이스 (타임아웃)
Revenue 재정의	비용 절감 · 안전 리스크 저감	가입 전환
접근 게이트	RBAC 4역할 × 6권한	AccessGate 2상태 × 4기능
골든셋	골든 퀘스천 + 환각 함정	골든 시나리오 5종
품질 지표	groundedness 100% · 환각률 0%	TTV ≤ 60초 · K-factor ≥ 0.3
테스트 하네스	FakeEmbedder / FakeStore / FakeLLM	Fake Auth / StyleApi / Analytics

ECB에서 Control이 비즈니스 규칙을 실행하고 Boundary가 외부와 통신한다면 — RBAC는 "누가 어떤 Control을 실행할 수 있는가"를 결정하는 접근 제어 레이어입니다

RBAC 4대 구성 요소

User 시스템 사용자 — 개발자·운영자·관리자

Permission 자원에 대한 행동 — read·write·delete

Role 권한 묶음 단위 — admin·developer·viewer

Resource 접근 대상 — DB·API·파일·서비스

도메인 특성은 체계의 구조를 바꾸지 않고 세 지점에서 파라미터로 흡수됩니다 — ① 계약의 내용 ② AARRR의 재정의 ③ 하네스의 Fake 대상. 13단계와 C2C 3요소는 두 도메인에서 누락 없이 동형적으로 유지되었습니다.

Claude Code · Codex 협업 — 하네스는 이식된다

2장 하네스 — 지도(GEMINI.md) + 설정 + 커맨드 + 금지 + 계약 ID — 는 도구 기능이 아니라 “규율”입니다. 파일 이름과 설정 키만 바뀔 뿐, 계약·테스트·금지는 그대로 이식됩니다.

① Gemini CLI

GEMINI.md · .gemini/settings.json

② Claude Code

CLAUDE.md · .claude/settings.json

③ Codex CLI

AGENTS.md · ~/.codex/config.toml

지도 파일	세 도구가 함께 읽게 하는 법
AGENTS.md	Codex 기본 인식 · Gemini는 settings.json context.fileName 에 추가
CLAUDE.md	@AGENTS.md 참조 또는 심링크(ln -s AGENTS.md CLAUDE.md)
계약·테스트·금지	docs/PRD.md · pytest -q — 세 도구 동일

승인·샌드박스 기본값은 도구마다 다릅니다 — 보수적으로 시작(read-only-plan-default)하고, push·rm·배포는 자동 승인하지 않습니다. 세부: docs.claude.com · developers.openai.com/codex.

AI 시대 개발자 역할의 재정의 — 그리고 한계

비교우위의 이동

AI가 계약→구현 구간의 비용을 극적으로 낮춘 상황에서, 인간의 비교우위는

- ① 상류 — 불변식을 발견·계약화한다
- ② 하류 — 계측을 설계해 개념의 실증 여부를 판정한다
- ③ 사슬 전체의 무단절성 감리

로 이동합니다. 개발자는 코드의 작성자에서 C2C 추적성의 설계자·감리자로 재정의됩니다.

사례에서 관찰된 노동 배분 — 개발자의 명시적 산출물은 대부분 계약 표·골든셋·실패하는 테스트·규칙 파일·프롬프트였고, 구현 코드의 상당 부분은 AI가 생성하되 계약 ID·테스트·규칙이 그 품질을 게이팅했습니다.

한계와 타당성 위협

- 통제 실험이 아닌 설계 과학적 시연·질적 교차 사례 분석
- 연구자가 설계에 참여한 실험 연구 — 제3자 재현 필요
- AARRR 역산의 전환율 가정은 도메인·시점에 따라 크게 변동
- 향후 — 계약으로부터 테스트·계측을 자동 생성하는 C2C 도구 체인

본 체계는 새 기법의 발명이 아니라 검증된 기법들의 목적론적 재조직입니다 — 기여는 이들을 C2C라는 단일 목표 아래 배치하고, 그 변환 규칙을 계약 ID와 매트릭스로 형식화한 데 있습니다.

CLOSING

“ 이 코드 한 줄은 어느 계약을 지키려는 것인가 ? ”

이 질문에 즉시 답할 수 있게 하는 것 — 그것이 C2C 입니다.

여러분은 이제 코드의 작성자가 아니라, AI 와 협업하며 C2C 추적 사슬을 설계하고 감리하는 사람입니다.

계약 ID 없는 동작은 구현하지 않는다

실패 로그가 최고의 컨텍스트다

테스트 약화는 즉시 거절한다

확장 뒤에는 수축을 둔다

구조 변경과 동작 변경을 섞지 않는다

매트릭스의 빈칸이 다음 질문이다